

Capturing packet with tcpdump

In this lab activity, I'll capture and analyze live network traffic using tcpdump. I'll use Linux commands in the Bash shell to complete these steps.

In addition , I used gemini to help me understand different flags used in an IP header.

Identify network interfaces

First I'll use the ifconfig command along with sudo to get elevated privilege to check the network environment

Here we have two interfaces , the eth0 interface and the lo interface.

```
sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1460
    inet 172.18.0.2  netmask 255.255.0.0  broadcast 172.18.255.255
    ether 02:42:ac:12:00:02  txqueuelen 0  (Ethernet)
    RX packets 1150  bytes 14038225 (13.3 MiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 756  bytes 63019 (61.5 KiB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 495  bytes 55800 (54.4 KiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 495  bytes 55800 (54.4 KiB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

I want to analyse the network traffic in the ethernet interface so I will use the command :

```
sudo tcpdump -i eth0 -v -c5
```

This command will run tcpdump with the following options:

interface option:

-i eth0: Capture data specifically from the eth0 interface.

verbose option :

-v: Display detailed packet data.

count option:

-c5: Capture 5 packets of data.

Here is the command and the first packet captured:

```
analyst@f63ecb2a76cc:~$ sudo tcpdump -i eth0 -v -c5

tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot
length 262144 bytes

11:29:02.169865 IP (tos 0x0, ttl 64, id 8024, offset 0, flags [DF],
proto TCP (6), length 146)

    f63ecb2a76cc.5000 >
nginx-us-central1-b.c.qwiklabs-terminal-vms-prod-00.internal.47436:

Flags [P.], cksum 0x58ff (incorrect -> 0xdfa7), seq
2889754074:2889754168, ack 1215609654, win 1056,
```

```
options [nop,nop,TS val 840878770 ecr 506029240], length 94
```

So here have have the timestamp : 11:29:02.169865

The IP version and its details : (tos 0x0, ttl 64, id 8024, offset 0, flags [DF], proto TCP (6), length 146)

tos 0x0: is the type of service for ipv4 that define the priority of the packet . it is the default values , among others

Hex Value	Technical Name (DSCP)	Common Usage	SOC Analyst Interpretation
0x00	Best Effort (BE)	Web, Email, Wix, YouTube	Standard: No special treatment. The most common value.
0x10	Low Delay	SSH, Telnet	Interactive: Needs fast response times so the user doesn't feel lag while typing.
0xb8	Expedited Forwarding (EF)	VoIP (Voice/Video calls)	Critical: Must skip the line to prevent audio cutting out.
0xc0	Internetwork Control	Routing protocols (OSPF, BGP)	Infrastructure: Essential for the network to stay alive.

TTL 64: It is a **hop counter**.

- **The Mechanism:** Every time a packet passes through a router (a "hop"), the router decreases the TTL value by **1**.
- **The Purpose:** It prevents packets from wandering forever in infinite loops if a network is misconfigured. If the TTL reaches **0**, the router drops the packet and sends an error message (ICMP Time Exceeded) back to the sender.

- **The Starting Value:** The sender sets the initial TTL. Different Operating Systems (OS) use different default values.

In the world of a SOC Analyst, we use the TTL for **OS Fingerprinting** (identifying the operating system without seeing it).

Initial TTL Value	Typical Operating System
64	Linux, Unix, FreeBSD, macOS, Android, iOS
128	Windows (all versions)
255	Network Infrastructure (Routers, Switches, Cisco)

id 8024:

Normal Situation: The "Librarian" Behavior

In a standard connection ,the Operating System acts like a librarian numbering pages in a book.

The Capture:

- Packet 1: ... id 8024, proto TCP ...
- Packet 2: ... id 8025, proto TCP ...
- Packet 3: ... id 8026, proto TCP ...

This is **Normal**. The numbers follow a perfect sequence (+1). This tells me the machine is healthy, there is no network congestion (no missing numbers), and no one is trying to interfere with the conversation.

2. Not Normal Situation: The "Identity Crisis" (Spoofing)

Imagine you are monitoring a conversation between two specific machines in your network.

The Capture:

- Packet 1: ... id 1200, proto TCP ...
- Packet 2: ... id 1201, proto TCP ...
- Packet 3: ... id 44982, proto TCP ... <-- **NOT NORMAL**
- Packet 4: ... id 1202, proto TCP ...

This is **Not Normal**.

The ID jump from 1201 to 44982 suddenly.

Then, Packet 4 goes back to 1202.

The Verdict: This is a classic sign of **Packet Injection**. A third party (the attacker) tried to "inject" a fake packet (Packet 3) into the conversation. Because the attacker couldn't guess the librarian's next number perfectly, they used a random ID, creating a "spike" in the sequence.

Summary

ID Sequence	Status	What it means
8024, 8025, 8026	Normal	Perfect communication.
8024, 8030, 8035	Suspicious	Packet loss (we are missing the ones in between).
8024, 55912, 8025	Critical Alert	Likely Attack/Injection (an outsider is talking).

offset 0: since we have the DF flag , because the packet is **never broken**, it remains a single, solid block of data. Offset will be 0.

Flag	Offset	Status	Analyst Conclusion
[DF]	0	Normal	Modern, healthy traffic. No fragmentation allowed.
[+] (More Frag)	0	Normal	This is the first piece of a broken packet.
[] (No flag)	> 0	Normal	This is a middle or end piece of a broken packet.

[DF]	> 0	Anomaly	Critical Alert. This is a "Forbidden Packet." Potential attack.
------	-----	---------	---

flags [DF]: it's the flag to say do not fragment the packet into smaller ones.

The Breakdown

DF (Don't Fragment): The **Standard**. Used by almost everything today to keep things fast, simple, and whole.

MF (More Fragments): The **Exception**. Used when a packet hits a "bottleneck" (older hardware, VPN tunnels, or satellite links) that can't handle the size.

The Risks:

Evasion: Malware "hides" in the pieces to sneak past a Firewall that isn't reassembling them.

DoS (Denial of Service): The server crashes because its memory is full of "unfinished puzzle pieces" waiting for the rest of the message.

Deep Analysis: The "Injection" Nuance

To be technically precise: **MF** doesn't just allow for "malware injection," it allows for **Signature Evasion**.

If an antivirus looks for the word "VIRUS", and you send:

Packet 1 (**MF** set): "VI"

Packet 2 (**MF** not set): "RUS" The security sensor sees "VI" (harmless) and "RUS" (harmless). Only the victim's computer sees the "VIRUS" once it glues them back together. **That is the true danger of the MF flag.**

proto TCP (6):

Technical Analysis: Protocol 6

In the IPv4 header, there is an 8-bit field called "Protocol." It tells the OS which "language" the data inside is speaking.

- **The Number 6:** This is the universal code for **TCP (Transmission Control Protocol)**.

- **The Job:** Unlike UDP (Protocol 17), which just "fires and forgets," TCP is **connection-oriented**. It ensures that every single byte sent is received in the correct order. If a piece is missing, TCP will automatically ask the sender to re-send it.

Protocol Number	Name	Security Persona
1	ICMP	The "Messenger" (Ping, Errors).
6	TCP	The " Accountant " (Careful, tracks everything).
17	UDP	The "Sprinter" (Fast, doesn't care about losses).

Normal Situation: The "Polite Conversation"

TCP is designed to be reliable and polite. A normal situation always follows the **Three-Way Handshake**.

The Scenario:

You are in your lab, and you click on a link to visit a website (Port 443).

How it behaves in **tcpdump**:

1. **SYN ([S]):** Your computer says: "Hello, I'd like to talk."
2. **SYN-ACK ([S.]):** The server says: "Hello! I'm ready. Did you hear me?"
3. **ACK ([.]):** Your computer says: "Yes, I heard you. Let's start!"
4. **DATA ([P.]):** Now, and only now, the actual data (the website) is pushed.

This is **Normal**. It's sequential, it's consensual, and it follows the "Rules of the Accountant."

Abnormal Situation: The "SYN Flood" (DDoS)

This is one of the most common attacks you will have to mitigate as a SOC Analyst.

The Scenario:

An attacker wants to crash a web server.

How it behaves in **tcpdump**:

- IP source.1234 > server.443: Flags [S], seq 1 ...
- IP source.1235 > server.443: Flags [S], seq 1 ...
- IP source.1236 > server.443: Flags [S], seq 1 ...
- *(Thousands of these per second, from different IPs)*

This is **Abnormal**. The attacker is sending the "Hello" ([S]) but **never sends the final "ACK."** The server stays in a "Half-Open" state, waiting for a response that never comes. It uses up all its memory waiting for these fake conversations, and eventually, the server crashes.

3. Abnormal Situation: The "Christmas Tree" Scan

This is a classic "hacker" move used for reconnaissance.

The Scenario:

An attacker is scanning your network to see which ports are open, but they want to be "sneaky" and bypass old firewalls.

How it behaves in **tcpdump**:

- IP attacker > server.80: Flags [FPU], seq 0 ...

This is **Extremely Abnormal**. The flags [FPU] stand for **FIN**, **PUSH**, and **URGENT** all at once.

In a normal world, you never "Finish" a conversation that hasn't started, while simultaneously saying it's "Urgent." It's called a "Christmas Tree" because the packet has every single light (flag) turned on.

- **The Goal:** The attacker is looking to see how the server reacts to "impossible" logic to identify the Operating System (Fingerprinting).
-

Summary Table

Situation	TCP Flags	Status	Analyst Conclusion
Handshake	[S], then [S.], then [.]	Normal	Healthy connection starting.
Data Flow	[P.] or [.]	Normal	Active communication.
Massive [S] only	[S] (no ACK)	Abnormal	SYN Flood Attack (DDoS).
Random [R]	[R] (Reset)	Suspicious	A connection was forcibly slammed shut. Could be a crash or a port scan.
Impossible Flags	[FPU] or [SR]	Abnormal	Malicious Scan or specialized attack.

length 146:

Technical Analysis: IP Length vs. Payload Length

In a packet capture, we see two different lengths.

Total Length (IP Header): This is the size of everything—the envelope + the letter.

Payload Length (TCP/UDP): This is what you see at the end of the line in `tcpdump`. It is just the letter inside.

The Calculation:

$\text{Total Length} - (\text{IP Header} + \text{TCP Header}) = \text{Payload Length}$

If your `length` is **146**, it means the application (like your web browser or a script) just sent exactly 146 bytes of information.

2. Normal Situation: "The Steady Flow"

In a healthy environment, the length should match the expected behavior of the protocol.

- **Small Lengths (e.g., 0 to 200):** These are usually "Control" packets. For example, an **ACK** packet (the "I heard you" message) often has a **length of 0** because it contains no data, just a confirmation flag.
- **A length of 146** is common for small HTTP requests, encrypted handshakes (TLS), or terminal commands (SSH).
- **Maximum Length (1460):** In most networks, the maximum payload is 1460 bytes. If you are downloading a large file from your Wix site, you will see thousands of packets with **length 1460** in a row. This is a **Normal** "Bulk Transfer."

3. Abnormal Situation: The "Heartbleed" or "Exfiltration"

As a SOC Analyst, the **length** field is how you detect "leaks."

A. The "Heartbleed" Style (Data Leak)

- **The Scenario:** You send a tiny request to a server, but the server responds with a very large length.
- **The Anomaly:** If you ask "Are you there?" (Length 1) and the server responds with **Length 1460** containing random memory data, you have a **Data Leak** vulnerability.

B. The "Reverse Shell" (Command & Control)

- **The Scenario:** You see a connection to an external IP.
- **The Anomaly:** Instead of big chunks of data, you see a constant stream of **length 1 to 50**.
- **The Interpretation:** This is "Not Normal" for a web browser, but it's **Normal for a Hacker** typing commands into a terminal. Every time they press a key, a small packet of a few bytes is sent. This is a sign of an active **Remote Access Trojan (RAT)**.

4. Summary Table

Length Value	Situation	Analyst Conclusion
0	ACK / SYN / FIN	Normal: Just a signal, no data.
1 - 500	SSH / Requests	Normal: Interactive traffic or commands.
1460	File Download	Normal: High-speed data transfer.

Unexpectedly Large	Ping / Heartbeat	Abnormal: Potential "Ping of Death" or Data Leak.
Constant Small Gaps	Web Browsing	Suspicious: Potential "Keylogging" or Shell activity.

Why exactly 146?

To understand this number, we have to look at the "Layer Cake" of the packet. In a standard Linux environment (like your eth0 interface), the headers have fixed sizes:

1. **IP Header:** 20 bytes (This contains the **tos**, **ttl**, **id**, etc.)
2. **TCP Header:** 20 to 32 bytes (This contains the ports and flags. Usually **32 bytes** if "TCP Options" like timestamps are used, which is common in modern Linux).
3. **The Payload (Data):** The actual message being sent.

The Math of your Packet:

If the **Total Length** is **146**:

- $\$146 \text{ (Total)} - 20 \text{ (IP Header)} = 126 \text{ bytes left for TCP}$.
- $\$126 - 32 \text{ (TCP Header with options)} = \mathbf{94} \text{ bytes of actual data}$.

2. Normal Situation: "The Typical Small Request"

A total length of 146 is perfectly **Normal**.

- **What it likely is:** This looks like a small "Keep-Alive" message, a tiny API call, or the start of an SSH encrypted exchange.
- **Why it's consistent:** Modern operating systems try to bundle small bits of data to be efficient. 146 bytes is a very common "size" for background system communication.

3. Abnormal Situation: "The Hidden Signal"

In a SOC, we look for **Length Inconsistencies**.

- **The Scenario:** You see a connection that *a/ways* has a total length of 146, every 30 seconds exactly.
- **The Anomaly:** Most human traffic (browsing Wix, typing in a terminal) has **variable** lengths.
- **The Verdict:** A perfectly fixed length of 146 at fixed intervals is a "Heartbeat." It usually means a **Botnet agent** or **Malware** is checking in with its Command & Control (C2) server to say "I'm still here, do you have orders for me?"

Practice Reading :

```
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size
262144 bytes
10:57:33.427749 IP (tos 0x0, ttl 64, id 35057, offset 0, flags [DF],
protot TCP (6), length 134)
    7acb26dc1f44.5000 >
nginx-us-east1-c.c.qwiklabs-terminal-vms-prod-00.internal.59788: Flags
[P.], cksum 0x5851 (incorrect > 0x30d3), seq 1080713945:1080714027, ack
62760789, win 501, options [nop,nop,TS val 1017464119 ecr 3001513453],
length 82
```

So for this packet , we used a `sudo ifconfig` to get the interfaces configurations, or `sudo tcpdump -D` if the first command is not recognised.

Here we have the name of the interface we are listening, which is the `eth0`. We could have multiple `eth` interfaces like `eth0`, `eth1`, `eth2` ...

```
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size
262144 bytes
```

```
link-type EN10MB (Ethernet)
```

This describes the **Data Link Layer (Layer 2)** protocol.

The Logic: "EN10MB" is a term that `tcpdump` uses for standard **Ethernet**. It doesn't mean the speed is limited to 10MB; it just tells the software to expect **Ethernet Headers** (MAC addresses) at the start of every frame.

If you were capturing on a Wi-Fi card or a VPN tunnel (Point-to-Point), this would change to `IEEE802.11` or `RAW`

```
capture size 262144 bytes
```

This is the **Snapshot Length**. It is the maximum amount of data `tcpdump` will "slice" from each packet.

The Logic: 262144 bytes is a massive amount (256 KB). Since a standard packet is only 1500bytes, this setting ensures you capture the **entire packet** (Headers + Full Data).

The Reality: The "1500 Rule" (MTU)

The **MTU (Maximum Transmission Unit)** is the physical limit of the "road" (the Ethernet cable/fiber).

- **The Law:** On a standard Ethernet network, no single IP packet can be larger than **1500 bytes**.
- **Why?** Because if a packet is too long and a tiny bit of static/interference hits the wire, the whole packet is ruined. Smaller packets mean less data is lost if there's a "glitch."
- **The Result:** If you have a 5,000-byte photo, your computer **must** slice it into 4 different packets (3 packets of 1500 and 1 packet of 500) before sending them.

The Logic: "Slicing" (DF) vs. "Chopping" (MF)

This is where your previous questions about the **DF flag** and **MF flag** connect to the Rule:

- **TCP Segmentation (Slicing):** Because your computer knows the "1500 Rule," it slices the data **at the source**. It puts the **[DF] (Don't Fragment)** flag on every slice. This is the "Normal" way.
- **IP Fragmentation (Chopping):** If a packet of 1500 bytes hits a "narrower road" (like a 1400-byte VPN tunnel), and it *doesn't* have the DF flag, the router chops it. This is the "Messy" way that causes security risks.

In what protocol would mf be used ? since tcp only use df

Protocol 17: UDP (The Primary Culprit)

As you suspected in your previous thought, **UDP (User Datagram Protocol)** is the #1 user of the **MF** flag.

- **The Reason:** UDP doesn't have a "negotiation" phase. If an application (like a video game or a streaming service) hands a 5,000-byte block of data to UDP, UDP doesn't know how to slice it. It just hands it to the **IP Layer**.
- **The Result:** The IP layer looks at the 1500-byte MTU rule and says, *"I have no choice but to chop this."* It creates fragments, sets the **MF** flag on the first few, and the receiver has to hope all the pieces arrive to put the puzzle back together.

Protocol 1: ICMP (The Diagnostic Tool)

Sometimes, **ICMP (Internet Control Message Protocol)**—the protocol used for **ping**—will use **MF**.

- **The Scenario:** If you run a command like **ping -s 5000 [IP_Address]**, you are forcing your computer to send a giant "Echo Request."
 - **The Result:** Since ICMP is simple and doesn't "segment" like TCP, it gets fragmented. You will see the **MF** flag in your capture for every piece except the last one.
-

The "Tunnel" Protocols (The Hidden Risk)

This is where things get "Cybersecurity heavy." Tunneling protocols often cause fragmentation even if the original data was TCP!

- **GRE (Protocol 47) and IPsec (Protocol 50/51):** * Imagine a 1500-byte TCP packet (with the **DF** flag).
 - It enters a **VPN tunnel**. The VPN adds its own 50-byte "wrapper" (header).
 - Now the packet is **1550 bytes**.
 - **The Conflict:** It's too big for the 1500-byte road. If the VPN is configured to allow it, it will **clear the DF flag and fragment the packet**, resulting in **MF** flags.
- **SOC Analyst Note:** We hate this. It slows down the VPN and makes it harder for our Firewalls to scan the traffic.

Deep Analysis: Why **MF** is a "Red Flag" today

In a modern, well-configured network in 2026, **you should almost never see the MF flag**. If you see **MF** in your SOC logs, I want you to immediately think of three possibilities:

1. **NFS (Network File System):** Old versions of NFS used giant UDP blocks.
2. **DNSSEC:** Modern, secure DNS can sometimes produce packets larger than 1500 bytes that get fragmented.
3. **An Attack:** An attacker is deliberately using **MF** to try and bypass your IDS with the "Signature Evasion"

Summary Table: DF vs. MF by Protocol

Proto col	Preferred Flag	Why?
TCP (6)	DF	Uses "Segmentation" to avoid the mess of fragmentation.
UDP (17)	MF	Doesn't know how to slice data; relies on IP to chop it.
ICMP (1)	MF	Only for large "pings" or error messages.

VPN Tunnel	MF (Sometimes)	When the "wrapper" makes the packet too big for the pipe.
-------------------	-----------------------	---

The Tool: The "262,144 Capacity" (Snaplen)

Now, why does `tcpdump` say its capacity is **262,144** if the rule says **1500**?

- **The "Just in Case" Principle:** Even though 99% of packets are ≤ 1500 , some special networks (Data Centers) use **Jumbo Frames** (9,000 bytes). Some internal computer traffic (Loopback) uses **65,535 bytes**.
- **The Security Margin:** As a SOC Analyst, if an attacker sends a "forbidden" packet that is 2,000 bytes long to try and crash a system, you want your tool to see the **whole thing**.
- **The Comparison:** If your tool's capacity was only 1500, and a 2000-byte "attack packet" arrived, your tool would be **blind** to the last 500 bytes.

By setting the capacity to **262,144**, `tcpdump` ensures it can catch the biggest "whale" in the ocean, even though it usually only catches "small fish" (1500 bytes).

4. Summary Table: The Rule vs. The Capacity

Feature	The Rule (MTU)	The Capacity (Snaplen)
Value	1500 Bytes	262,144 Bytes
Where?	The Network Road (Ethernet)	The Analyst's Tool (tcpdump)
Why?	To prevent data corruption on cables.	To ensure the tool never "chops" a packet.
In a SOC...	We expect packets to follow this.	We rely on this to see the full "truth."

```
10:57:33.427749 IP (tos 0x0, ttl 64, id 35057, offset 0, flags [DF],  
protot TCP (6), length 134)
```

```
10:57:33.427749
```

This is the timestamp.

We have the IP header with its different flags:

- tos (type of service) that define its priority level which is 0x0 default in this case
- ttl time to live , here 64 represent linux os , android, ios or other
- id 35057